Finding Risky Capabilities in Open-Source AI Agents: A Lightweight Static Scanner and Its Comparison with Manual Review

Riya Jamsutkar
riya.jamsutkar@vit.edu.in

Shubham Patel
shubham.patel@vit.edu.in

Atharv Londhe
atharv.londhe@vit.edu.in

Department of Computer Engineering, Vidyalankar Institute of Technology, Mumbai, India

Abstract—AI agents now call tools, run commands, write files and reach external services, yet the usual software bills of materials only list libraries. This project asks a small, practical question: can a simple static scanner find risky agent capabilities in source code, and how closely does it agree with a careful manual review? We built a Python scanner that uses abstract syntax tree (AST) rules and keyword patterns for six capabilities: shell or code execution, file writing, external network access, credential use, persistent state, and human-approval steps. We ran it on 10 open-source agent repositories (2,414 Python files, about 574,000 lines) and compared its output with a manual review of the same code. The scanner agreed with the manual labels on 55 of 60 repository–capability decisions (91.7%), with macro precision 0.90, recall 1.00 and F1 0.95. All five errors were false positives caused by example code and by input() prompts that are not approval gates. However, a baseline that simply flags every capability scores F1 0.87, because six of the ten repositories have all six capabilities. An SBOM from Syft, read for package-name clues, can hint at only three of the six capabilities and reaches F1 0.34. We conclude that a simple scan adds capability information that SBOMs miss, but that repository-level presence is too coarse to separate agents, and that approval steps need finer, location-level analysis.

Index Terms—AI agents, software bill of materials, static analysis, security, tool use, capability analysis

I. INTRODUCTION

Modern AI agents do more than answer questions. They can read private documents, run shell commands, edit files, call web APIs and, in some designs, hand work to other agents [1], [2]. This makes them useful, but it also means that a mistake or a malicious prompt can turn into a real action on a real system [3].

Software bills of materials (SBOMs) such as SPDX and CycloneDX list the components inside a program, and they have become a standard way to track supply-chain risk [4], [5]. They tell us which packages an agent depends on. They do not tell us what the agent is allowed to do at run time. Dutta and Moharir describe this as an "agentic transparency gap" and propose AgentRiskBOM, a JSON-schema artifact that records runtime authority such as tool permissions, memory and approval gates [6].

Their work focuses on defining a rich schema. In this project we take a smaller and more hands-on view. Before anyone fills in such a document, someone has to find out what an agent can actually do, and doing that by hand for every repository does not scale. We therefore study whether a lightweight scanner can do part of that job.

The contributions of this paper are:

1) a six-capability taxonomy for agent code that is easy to apply by hand or by script;

2) an open, rule-based Python scanner that detects these capabilities and logs the file and line behind every flag;

3) an empirical comparison of the scanner against manual review, a flag-everything baseline and a standard SBOM tool on 10 repositories, including an error analysis.

The rest of the paper is organised as follows. Section II reviews related work, Section III explains our method, Section IV reports results, Section V discusses them and their limits, and Section VI concludes.

II. RELATED WORK

A. Bills of Materials for Software and AI Systems

SPDX and CycloneDX describe software components, versions and licences [4], [5]. The NTIA minimum-elements guidance made SBOMs a common requirement in supply-chain security [7]. Extensions for machine learning, often called AIBOM or MLBOM, add model and dataset information, but they mainly describe what an AI system is made of rather than what it can do [6].

B. Agentic Systems and Their Risks

Frameworks that combine language models with tools, such as ReAct-style agents, let a model decide which action to take next [1], [2]. Work on indirect prompt injection shows that text read from the web or from files can steer such an agent into unintended actions [3]. Community guidance such as the OWASP list of risks for LLM applications also highlights excessive agency and insecure tool use [8].

C. AgentRiskBOM

AgentRiskBOM adds fields for autonomy, tool permissions, memory, credential scope, approval gates, audit signals and

inter-agent communication on top of existing BOMs [6]. The authors build a corpus of 13 open-source agents and evaluate schema validity, category coverage and change detection. Our work is complementary. We do not propose a new schema, and we did not run their tooling. Instead, we test how much of the needed capability information can be recovered automatically from source code.

D. Static Analysis for Security

Static analysers such as Bandit scan Python code for risky calls like shell execution and weak cryptography [9]. These tools are built for general software and report code-level issues, not agent-level capabilities such as "has a human approval step". Our scanner reuses the same basic idea but targets agent-specific capabilities.

III. METHODOLOGY

A. Repository Selection

We selected 10 popular open-source Python agent repositories from GitHub covering four archetypes: coding agents (3), research or retrieval-augmented agents (1), multi-agent and orchestration frameworks (4) and general task agents (2). Table I lists them. Each repository was cloned with depth 1 in October 2026 and pinned to the commit shown. One repository (Open Interpreter) is pinned to its last Python release, v0.4.2, because its current default branch no longer contains an agent written in Python. We did not use the repositories from the AgentRiskBOM corpus, since we did not have its list.

TABLE I. REPOSITORIES STUDIED

Repository	Commit	Type	Files	LOC
MetaGPT	11cdf46	Multi-agent	581	66,221
aider	5dc9490	Coding	106	25,900
babyagi	fa8930e	General	41	6,153
crewAI	753d192	Multi-agent	951	324,033
smol-ai/developer	a6747d1	Coding	13	1,199
gpt-researcher	3582a68	Research/RAG	217	26,038
langgraph	799c2f0	Multi-agent	259	85,325
Open Interpreter	13061d2	Coding	140	18,026
smolagents	96f33fa	General	48	17,673
swarm	6af0b4c	Multi-agent	58	3,434
Total			2,414	574,002

Files and lines count Python files outside test folders.

B. Capability Taxonomy

We defined six capabilities, chosen because each one changes what damage an agent could do if it is misled. Table II lists them with the code signals the scanner looks for. "Network access" means calls to services other than the model provider's own API, since nearly every agent calls a model.

TABLE II. CAPABILITIES AND SCANNER SIGNALS

Capability	Signals searched in code
Shell/code execution	subprocess.run/call/Popen/check_output, os.system, os.popen, eval, exec
File writing	open() in write/append mode, write_text, shutil, os.remove
Network access	requests, httpx, aiohttp, urllib, socket
Credential use	os.environ, os.getenv, dotenv, getpass
Persistence	sqlite3, sqlalchemy, chromadb, faiss, redis, pymongo, pickle.dump
Approval step	input(), human_input, require_approval, interrupt_before, (y/n)

C. Scanner Design

The scanner walks every Python file in a repository, skipping test and hidden folders, and parses it into an AST. It marks a capability as present if it finds a matching import or function call (Table II). Approval steps are found with a regular-expression search because frameworks name them in many different ways. The output is one row per repository with a 0/1 flag per capability, plus a log of the file, line and trigger behind each flag. The scanner is about 130 lines of Python and uses only the standard library. Its rules were written before it was run and were not changed afterwards, even where the review (Section IV) showed gaps.

D. Manual Labelling

To create a reference answer, we reviewed the code of each repository and recorded whether each capability is present in shipped agent code, meaning code that is part of the library, CLI or bundled application. Capabilities that appear only in tests, documentation, examples, evaluation scripts or build scripts do not count. For every flagged case we opened the triggering files and checked the context. For every case the scanner did not flag, we ran broader searches (for example aliased imports, other HTTP or database libraries, and non-Python files). Where a repository had many hits in clearly relevant files, we confirmed a sample of them rather than every hit. An approval step means a gate that must be passed before an action runs; a prompt that only reads the user's next chat message does not count. Labelling was done by a single reviewer with AI assistance for code navigation; no second reviewer was available, so agreement could not be measured.

E. Baselines and Metrics

We compared the scanner with two baselines. The first is a conventional SBOM generated with Syft v1.54.1 in CycloneDX JSON format [10]. Because an SBOM lists packages and not behaviour, we read it for package-name clues: a network package (requests, httpx, aiohttp, urllib3) counts as evidence of network access, a database or vector-store package (sqlalchemy, chromadb, faiss, redis, pymongo) as persistence, and python-dotenv or keyring as credential use. The other three capabilities cannot be inferred from package names. The second baseline simply flags every capability in every repository. We did not run AgentRiskBOM itself; we only discuss its field coverage [6].

For each capability we count true positives (TP), false positives (FP) and false negatives (FN) against the manual labels and compute precision P = TP/(TP+FP), recall R = TP/(TP+FN) and F1 = 2PR/(P+R). We report the macro average over the six capabilities.

IV. RESULTS

A. Dataset and Reference Labels

Manual review found 47 present capabilities out of 60 repository–capability pairs (78%). Shell or code execution is present in 8 repositories, file writing in 9, network access in 8, credential use in 9, persistence in 7 and approval steps in 6. Six of the ten repositories have all six capabilities. Only swarm has none, and smol-ai/developer has two (file writing and credential use).

B. Scanner Accuracy

Table III shows the scanner results against the manual labels. Overall the scanner matched 55 of 60 decisions (accuracy 91.7%; 47 TP, 5 FP, 0 FN, 8 TN).

TABLE III. SCANNER RESULTS PER CAPABILITY

Capability	TP	FP	FN	P	R	F1
Shell/code exec.	8	0	0	1.00	1.00	1.00
File writing	9	1	0	0.90	1.00	0.95
Network access	8	0	0	1.00	1.00	1.00
Credential use	9	0	0	1.00	1.00	1.00
Persistence	7	2	0	0.78	1.00	0.88
Approval step	6	2	0	0.75	1.00	0.86
Macro average				0.90	1.00	0.95

The scanner produced no false negatives, but it produced five false positives, all in swarm and smolagents. Three came from example code that the scanner does not exclude: a file write and an sqlite3 import in swarm's examples folder, and a sqlalchemy import in a smolagents example. The other two came from the approval rule. The input() call in swarm's REPL only reads the user's next chat message, and the input() call in smolagents' user-input tool asks the user a question without gating any action.

Detection was also not always right for the right reason. Of the 47 true positives, three were reached through an unrelated trigger. For gpt-researcher, the shell flag came only from evaluation scripts, while the real calls in the main code use subprocess.check_call, which is not in our rule set. Its persistence flag came from a pickle call that stores browser cookies, not from its report storage. For Open Interpreter, the persistence flag came from an sqlite3 call that reads the macOS Messages database, while its real persistence is a saved conversation history. These are cases where the repository-level answer was correct but the evidence was not.

C. Comparison with Existing Approaches

Table IV compares the approaches on the same 60 decisions.

TABLE IV. COMPARISON OF APPROACHES (MACRO AVERAGE)

Approach	P	R	F1	Detectable (of 6)	Manual effort
Flag everything	0.78	1.00	0.87	6	None
Syft SBOM + package hints	0.47	0.27	0.34	3	None
Our scanner	0.90	1.00	0.95	6	Low

The SBOM approach found some evidence for network access (5 of 8 repositories), credential use (4 of 9) and persistence (4 of 7). It gave no evidence for shell execution, file writing or approval steps, which no package name reveals. Syft also returned zero components for swarm, which ships no dependency manifest, and only 12 to 22 components for babyagi, gpt-researcher and smolagents. The AgentRiskBOM schema has fields for all six of our capabilities by design (tool permissions, credential scope, memory and approval gates) [6], but it records them through a structured document and we did not measure how it is filled in.

V. DISCUSSION AND LIMITATIONS

The most important result is a warning about the scanner's headline score. Its F1 of 0.95 is only 0.08 above the flag-everything baseline of 0.87, because most agent repositories simply have most of the capabilities. Repository-level presence is therefore a weak signal for telling agents apart. The scanner does earn its advantage on the cases that matter: it correctly said "no" for 8 of the 13 absent capabilities, it flagged only the two capabilities that smol-ai/developer really has, and it clearly beats SBOM-based hints. A more useful next step would be to report where a capability lives (which module, and whether it is reachable from the default agent loop) instead of a single yes or no. The evidence log produced by our scanner is a first step in this direction.

The error analysis points to two concrete improvements: excluding example, documentation and evaluation folders, which would remove three of the five false positives, and replacing the plain input() rule with a check for confirmation prompts that are followed by an action. We did not apply these changes, to keep the reported results free of tuning on the test set. The check_call gap shows that a fixed list of function names is fragile.

This study has several limits. First, the dataset is small (10 repositories) and was chosen from popular projects, so the numbers are indicative and not general rates. Second, the labels come from a single reviewer working with AI assistance, and for large repositories we verified a sample of hits and not every one, so some labels may be wrong. Third, the scanner reads only Python; plug-ins, configuration files and other languages are invisible to it. Fourth, finding a risky call does not prove that the agent can reach it at run time, so the scanner reports possible capability, not granted permission. Fifth, the SBOM baseline depends on our own package-name mapping, which is a judgement call and could

be drawn more generously or more strictly. Finally, the capability definitions, especially for approval steps and network access, involve choices that other reviewers might make differently. Future work could add data-flow analysis, support for more languages, a second reviewer, and an export of scanner results into the AgentRiskBOM schema [6].

VI. CONCLUSION

We tested whether a simple static scanner can recover agent capability information from source code. On 10 repositories it matched manual review on 91.7% of decisions and reached a macro F1 of 0.95, clearly above an SBOM-based approach (0.34) but only modestly above flagging everything (0.87), because most agents have most capabilities. The scanner is not a replacement for careful review, but it gives a fast first pass and an evidence trail that can make richer documents such as AgentRiskBOM easier to fill in. The scanner, labels and raw outputs are provided as supplementary material.

REFERENCES

[1] S. Yao et al., “ReAct: Synergizing reasoning and acting in language models,” arXiv:2210.03629, 2022.

[2] T. Schick et al., “Toolformer: Language models can teach themselves to use tools,” in Proc. NeurIPS, 2023.

[3] K. Greshake et al., “Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection,” arXiv:2302.12173, 2023.

[4] SPDX Workgroup, “The Software Package Data Exchange (SPDX) specification,” Linux Foundation. [Online]. Available: https://spdx.dev

[5] OWASP CycloneDX, “CycloneDX bill of materials specification.” [Online]. Available: https://cyclonedx.org

[6] S. Dutta and A. K. Moharir, “AgentRiskBOM: A risk-scoping security bill of materials for agentic AI systems,” arXiv:2606.21877, Jun. 2026.

[7] U.S. Dept. of Commerce and NTIA, “The minimum elements for a software bill of materials (SBOM),” Jul. 2021.

[8] OWASP Foundation, “OWASP top 10 for large language model applications.” [Online]. Available: https://owasp.org

[9] PyCQA, “Bandit: Security linter for Python code.” [Online]. Available: https://github.com/PyCQA/bandit

[10] Anchore, “Syft: CLI tool and library for generating a software bill of materials,” v1.54.1. [Online]. Available: https://github.com/anchore/syft